

Ames Housing Dataset

Regression problems

Outlier removal develops better regressive models

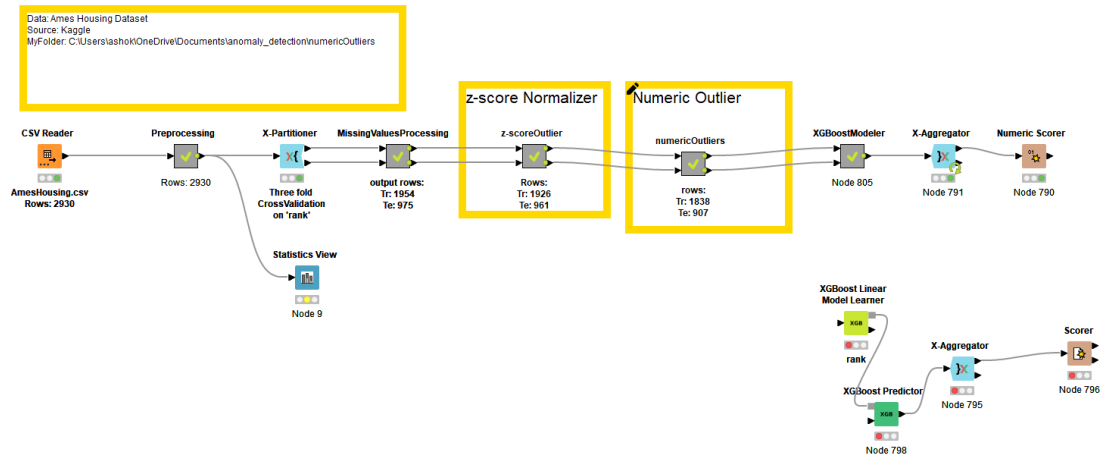
Last amended: 15th Sep, 2026
My folder: C:\Users\ashok\OneDrive\Documents\anomaly_detection\numericOutliers
Virtual Machine: ubuntu_database
#GitHub site:
#KNIME Files:
 a) outlierRemoval_ImprovingXgboost-I
 b) outlierRemoval_ImprovingXgboost-II
 c) outlierRemoval_ImprovingXgboost-III
Ref: [This link Here](#)

Contents

Approach	3
About the dataset	3
KNIME Writing data in a node:	3
Start from here:	3
Chapter-I	5
1. Column name replacer	5
2. Resort column names for easy search:	5
3. Rule Engine.....	6
4. Column Filter	7
5. Metanode creation	7
Chapter-II	9
Chapter-III.....	10
Chapter-V	14
Chapter-VI	16
Chapter VII.....	17
Chapter VIII.....	19
Salary Dataset.....	20
More Datasets.....	20

This example shows how data pre-processing and outlier removal can increase predictive performance in a regressive model. We predict *SalePrice* in the [AmesHousing](#) dataset, in a step-by-step manner.

Out final model is as follows:



Result with 10-fold cross-validation:

Statistics - 7:790 - Numeric Scorer	
File	
R ² :	0.888
Mean absolute error:	13,940.946
Mean squared error:	386,402,149.316
Root mean squared error:	19,657.114
Mean signed difference:	-90.352
Mean absolute percentage error:	0.094
Adjusted R ² :	0.888

Approach

In a number of datasets for regression the target feature may have skewed distribution. Along with this, an important feature within the dataset may also have skewed distribution. In such cases, the approach of outlier removal as illustrated in this example may benefit predictive modelling.

About the dataset

The Ames Housing Dataset is a well-known dataset in the field of machine learning and data analysis. It contains various features and attributes of residential homes in Ames, Iowa, USA. The dataset is often used for regression tasks, particularly for predicting housing prices.

Here are some key details about the Ames Housing Dataset:

- **Number of Instances:** The dataset consists of 2,930 instances or observations.
- **Number of Features:** There are 79 different features or variables that describe various aspects of the residential properties.
- **Target Variable:** The target variable in the dataset is the "SalePrice," representing the sale price of the houses.
- **Data Types:** The features include both numerical and categorical variables, covering a wide range of aspects such as lot size, number of rooms, location, construction, and more.

The Ames Housing Dataset is widely used in the machine learning community for tasks such as regression modeling, feature engineering, and predictive analytics related to housing prices. It serves as a valuable resource for developing and testing machine learning algorithms and techniques in the real estate domain.

KNIME Writing data in a node:

To write data inside a node, proceed as follows:

Proceed as follows:

```
Add and open your 'Table Writer' node's configuration.  
Delete everything inside the file path text box.  
Paste exactly: knime://knime.workflow/data/my_output_data.tableClick  
OK and execute the node.
```

To read the data, add Table Reader and point exactly to this path and file. Done.

Start from here:



Chapter-I

Data Preprocessing

Go on adding these few pre-processing nodes. The nodes used are:

- i) *Column Name Replacer*, ii) *Column Resorter*, iii) *Rule Engine* and
- iv) *Column Filter*.

1. Column name replacer

Remove spaces from all column names.

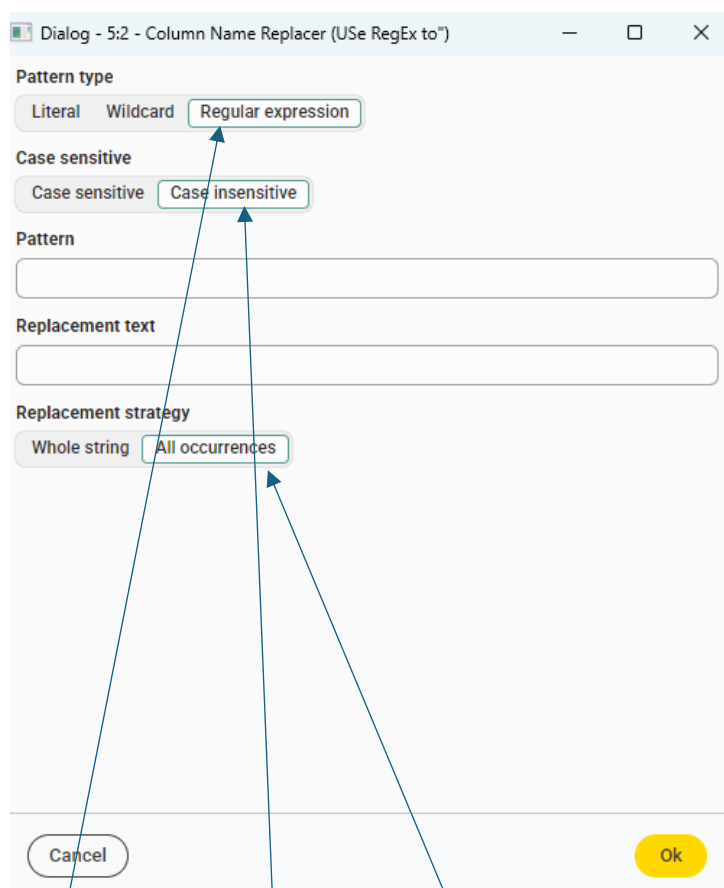


Figure 1: Select Regular Expression, Case Insensitive and All occurrences as here. Write a space in Pattern and nothing in Replacement Text. Click OK.

2. Resort column names for easy search:

We have a large number of columns. To quickly search them in various nodes, better they are listed in a sorted manner.

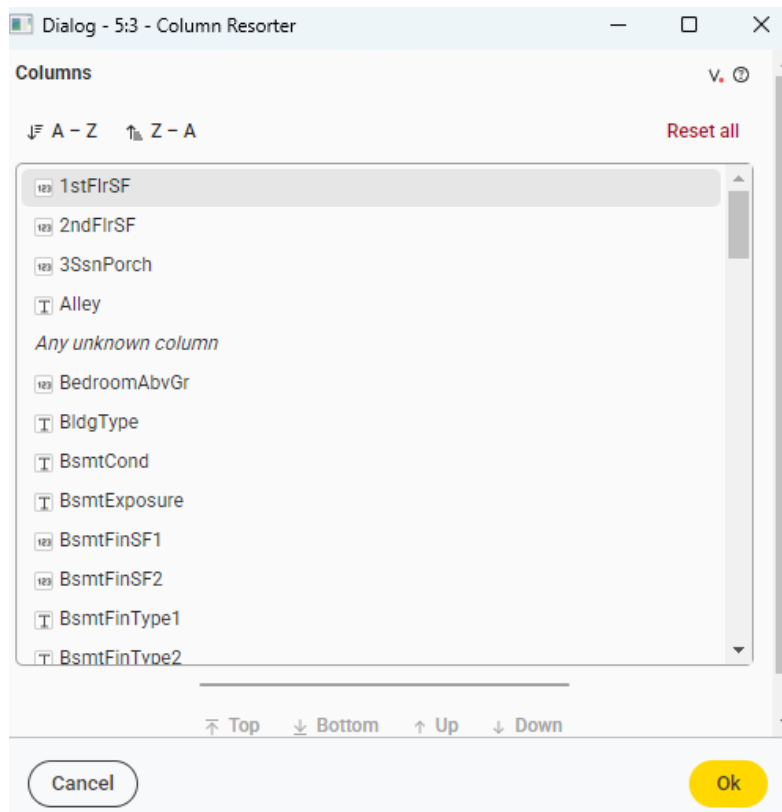


Figure 2: Click A-Z and click OK to order column names.

3. Rule Engine

Add a *Rule Engine* with to replace column 'OverallCond' with binary values and transform it into categorical feature. Call new node as 'rank'. We will also test it as our another Target node for classification.

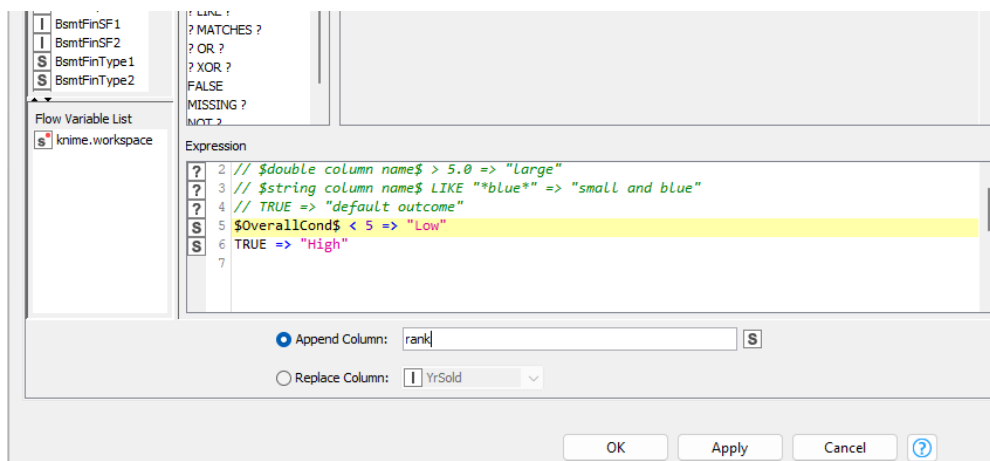


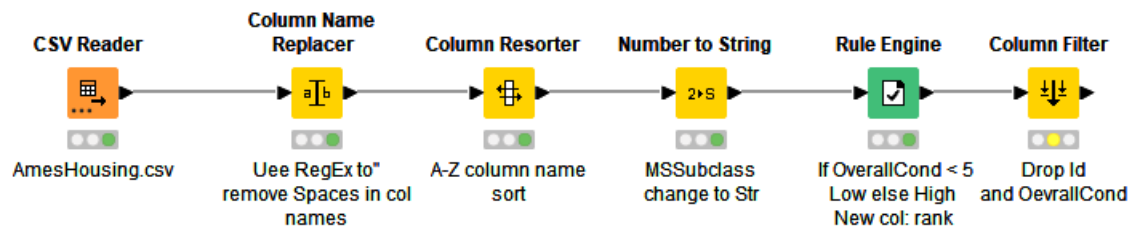
Figure 3: Name the new column as 'rank'. It is appended to existing columns

We name it as 'rank' column as our target for classification.

```
$OverallCond$ < 5 => "Low"
TRUE => "High"
```

4. Column Filter

Finally, we add a column filter node to filter out 'OverallCond', Id and 'MSSubclass' columns. 'MSSubclass' has high correlation with 'rank'.



5. Metanode creation

We transform or bundle all but one starting node into *Preprocessing metanode*. Just select your nodes, right-click and click *Create Metanode*. Give it a name and you are done. If the output port is missing, proceed as below.

To add an output port to *Preprocessing* node, right-click on it. In the context menu click: *Metanode* → *Reconfigure* → *Add an output port for Table*, as below:

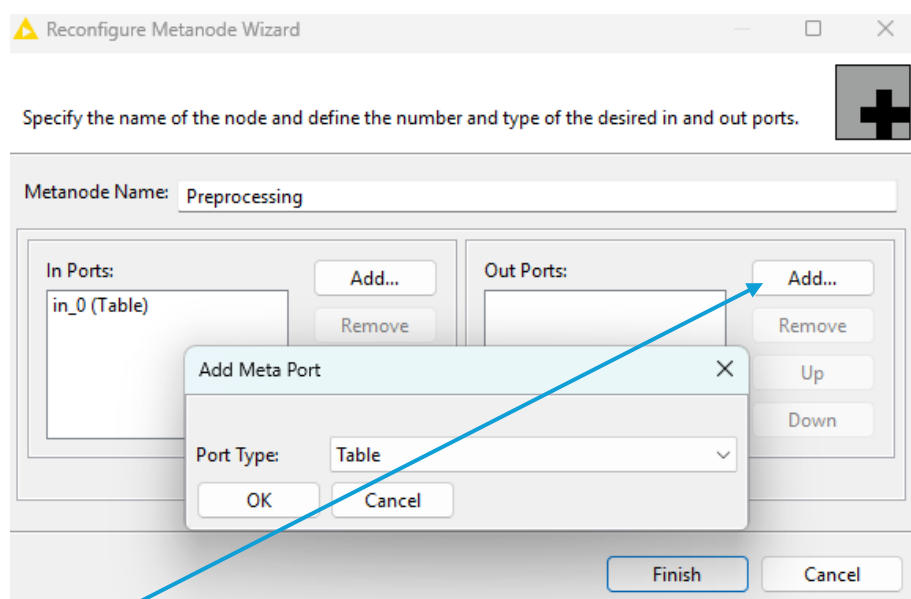


Figure 4: Click on Add button to add a Table port.

Then open *Preprocessing Metanode* and join the output port of the last node to input in the right wall. So, we do:

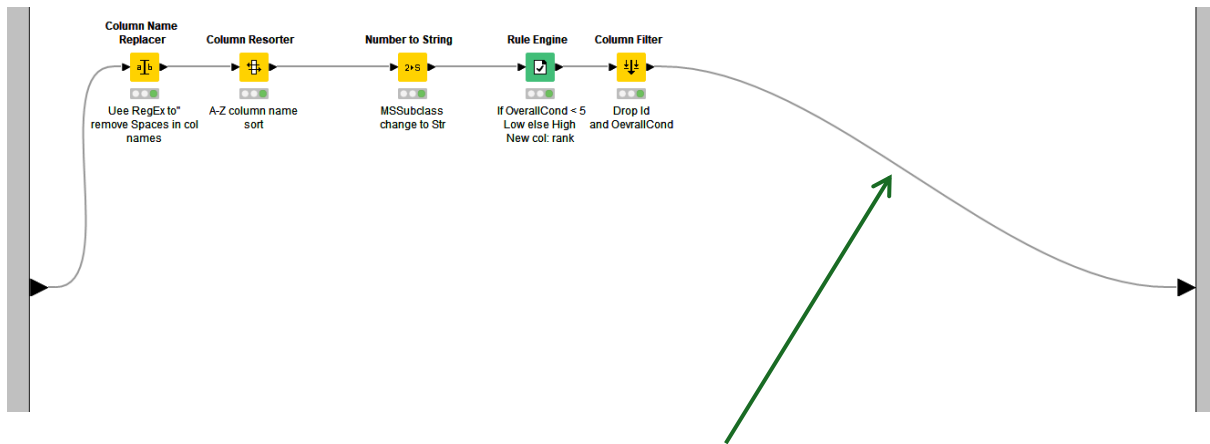


Figure 5: Right click to open Preprocessing mode and connect the last node's output port to the port on the right-wall as below.

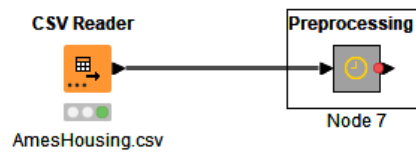


Figure 6: Preprocessing node now has an output port.

Chapter-II

Cross-validation node

We will be using three-fold cross-validation to evaluate our models. So, we add a Cross-validator (Cross-Partitioner: **X-Partitioner**) as below.

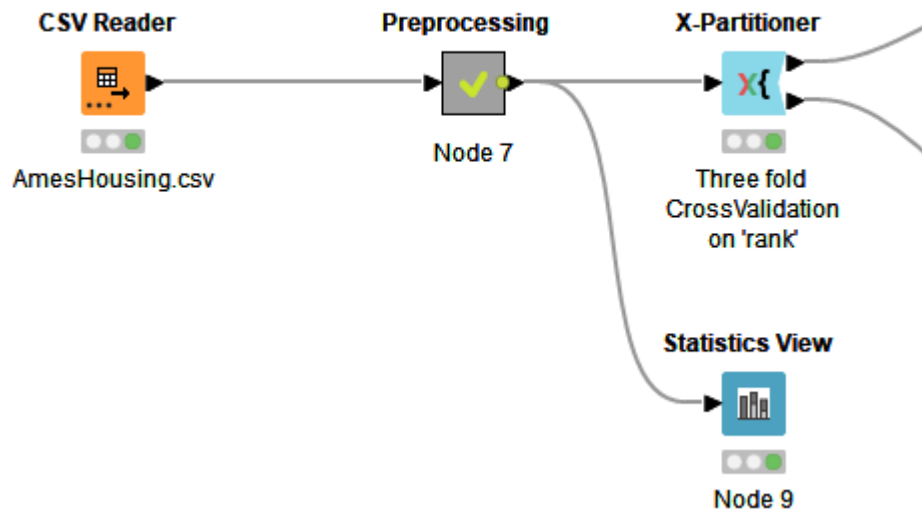


Figure 7: Try with three-fold, five-fold and 10-fold cross-validation.

Perform:

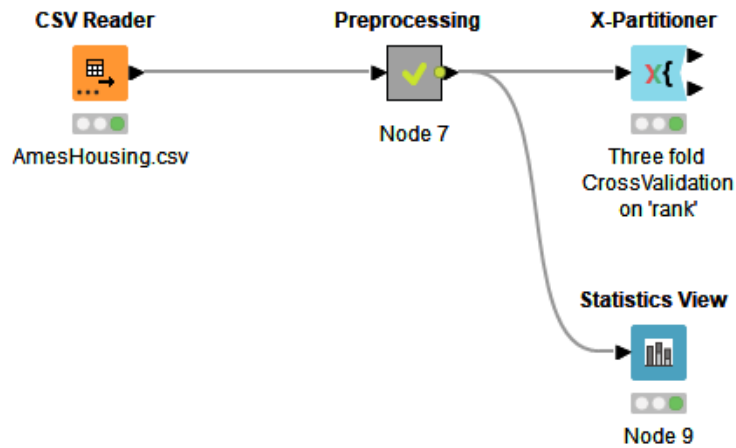
Random sampling:	If target is SalePrice
Stratified sampling:	if target is rank

Chapter-III

Missing values handling

File: outlierRemoval_ImprovingXgboost-I

Add *Statistics View* node:



Configure '*Statistics View*' node first for columns of data type Number:

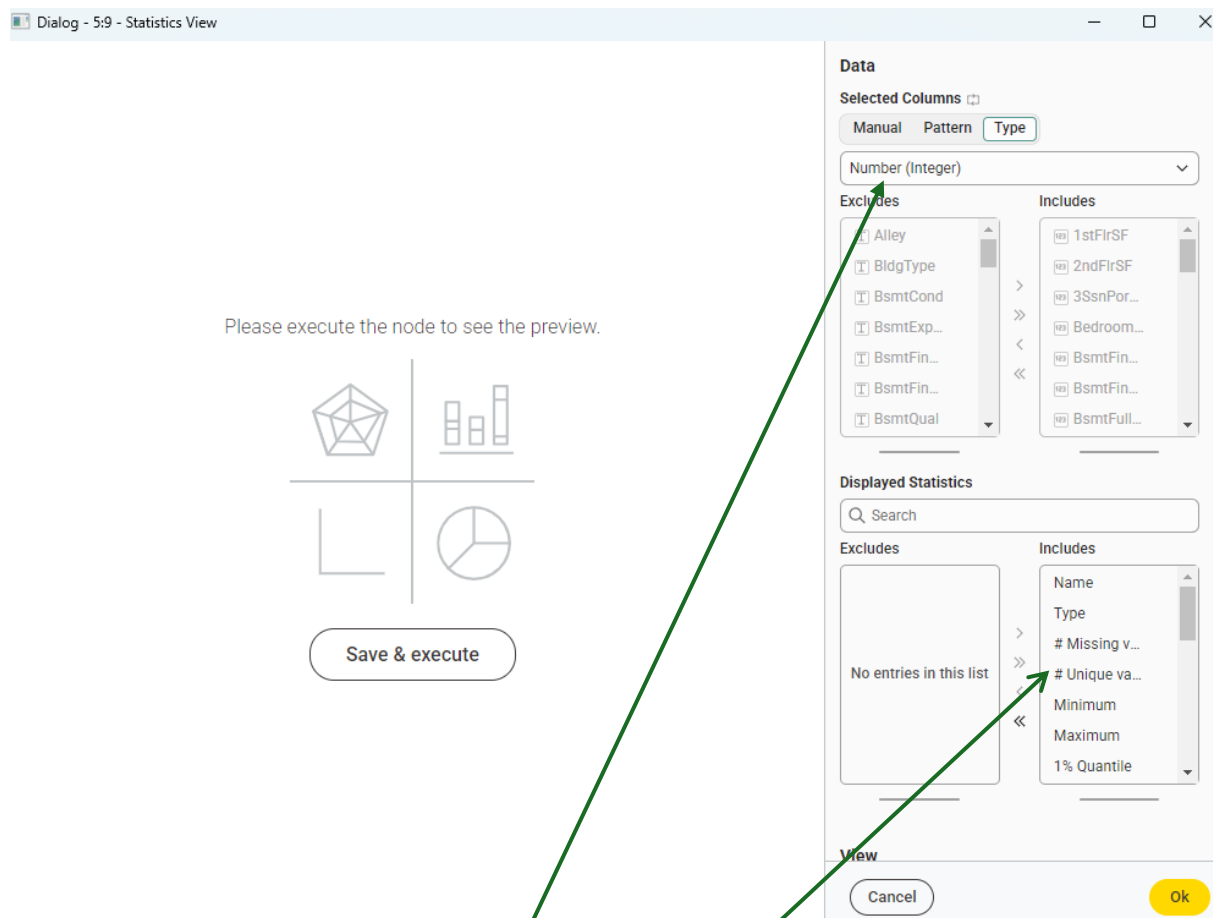


Figure 8: Configure first for columns with Type->Number and we select all statistics. Click OK.

In the Statistics View, sort 'Missing values' column in Descending order. To decide how to fill up missing values, observe 50% quartile (median) and mean in each case. Here is the brief:

Name	Type	# Missin... ↓	# Unique val...	Minimum	Maximum	1% Quantile	5% Quantile	10% Quantile	25% Quantile	50% Quantile...	75% Quantile	90% Quantile	95% Quantile	99% Quantile	Mean
LotFrontage	Number (Intege	490	128	21	313	21	32	43	58	68	80	95	107	136.59	69.225
GarageYrBlt	Number (Intege	159	103	1,895	2,207	1,915	1,928	1,941.2	1,960	1,979	2,002	2,006	2,007	2,009	1,978.132
MasVnrArea	Number (Intege	23	445	0	1,600	0	0	0	0	0	164	324.6	467.2	771.92	101.897
BsmtFullBath	Number (Intege	2	4	0	3	0	0	0	0	0	1	1	1	2	0.431

Figure 9: Top four columns with missing values.

There is not much difference between Mean and Median for the first three cases where number of missing values is high.

Name	# Missing va... ↓	50% Quantile (M...	Mean
LotFrontage	490	68	69.225
GarageYrBlt	159	1,979	1,978.132
MasVnrArea	23	0	101.897
BsmtFullBath	2	0	0.431
BsmtHalfBath	2	0	0.061

Figure 10: For the first three columns values of Mean and Median are about the same.

For the rest Number columns, we can select Median as the missing value filler. And, for the columns with String data type, the status of missing values is as below:

Statistics

Rows: 45 | Columns: 4

Name	# Missing values ↓
MasVnrType	23
BsmtExposure	4
BsmtFinType2	2
GarageFinish	2

Figure 11: Columns with string data type—Missing values

In these cases we select Mode as the filler.

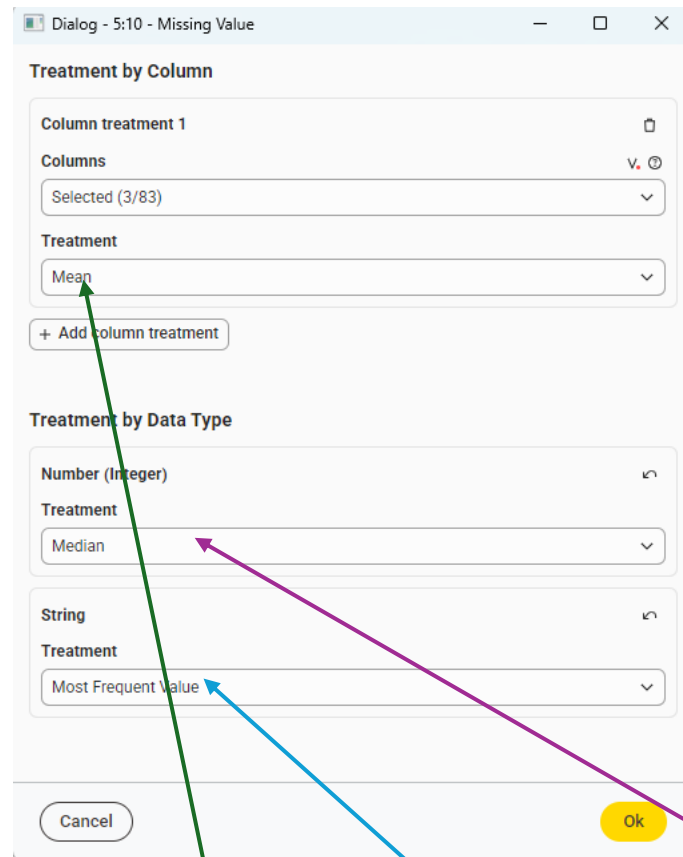


Figure 12: For three columns, we select Mean as the filler. For the remaining number type, we select Median as the filler and for all String types we select Mode as the filler.

In KNIME diagram, we add *Missing Value* node to analyse each feature on training side of data. And then, apply the same missing-value-fill-policy on the test side.

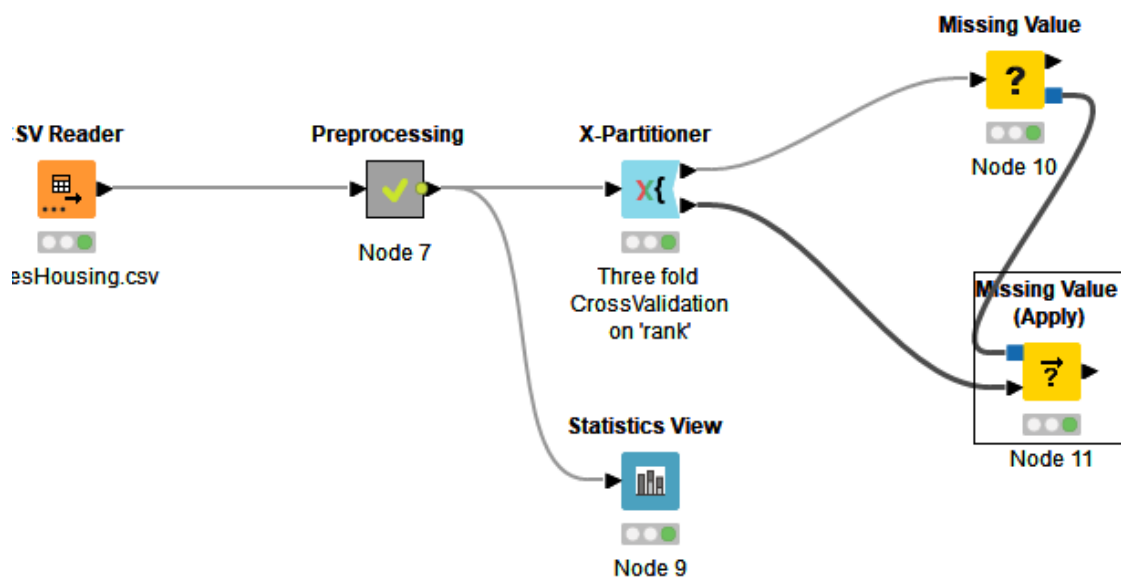


Figure 13: We will shortly put both the Missing value nodes in a Metanode.

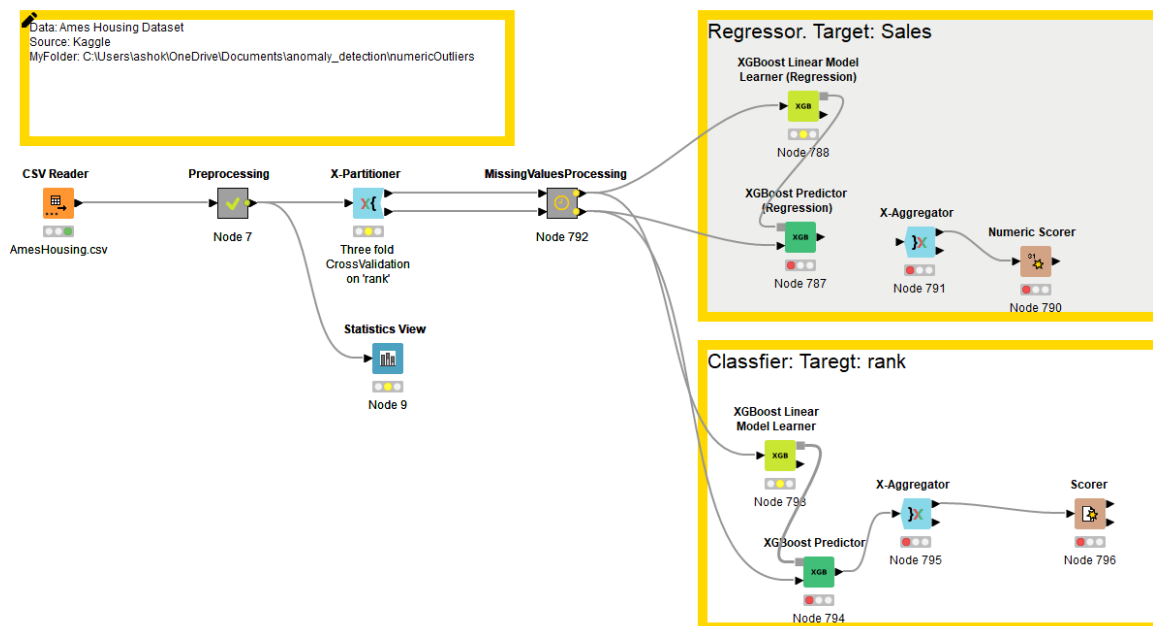
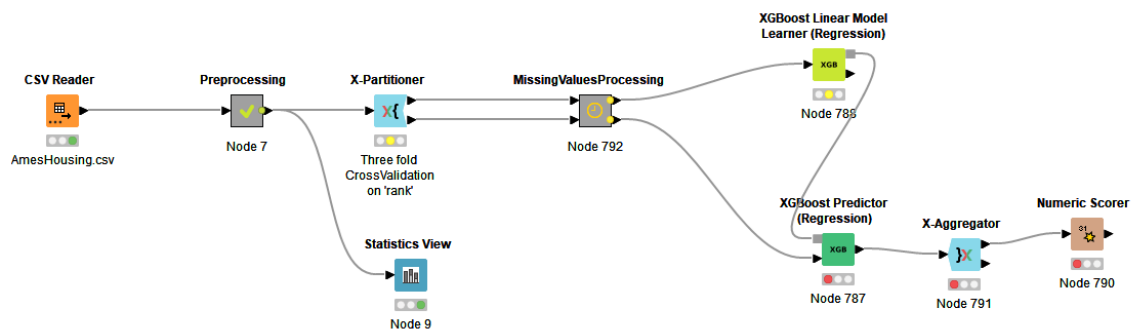


Figure 14: Complete Configuration for two kinds of cases: Regression and Classification



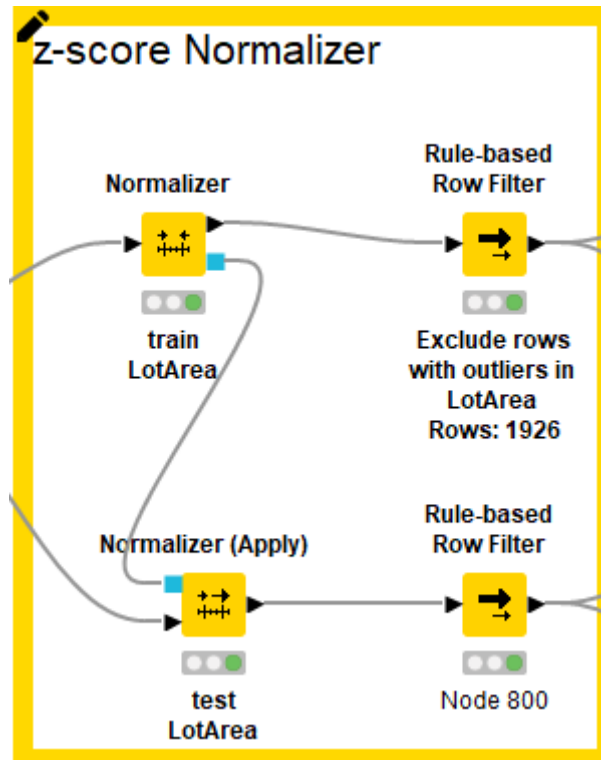
Result is as below for 10-fold cross-validation:

Statistics - 6:790 - Numeric Scorer	
File	
R ² :	0.852
Mean absolute error:	17,668.185
Mean squared error:	945,261,686.918
Root mean squared error:	30,745.108
Mean signed difference:	127.155
Mean absolute percentage error:	0.111
Adjusted R ² :	0.852

Chapter-V

Remove outliers

We now remove outliers first from LotArea and then from Salesprice. We use Z-score to remove outliers from LotArea. The basic configuration is as follows:



Normalizer is configured, as:

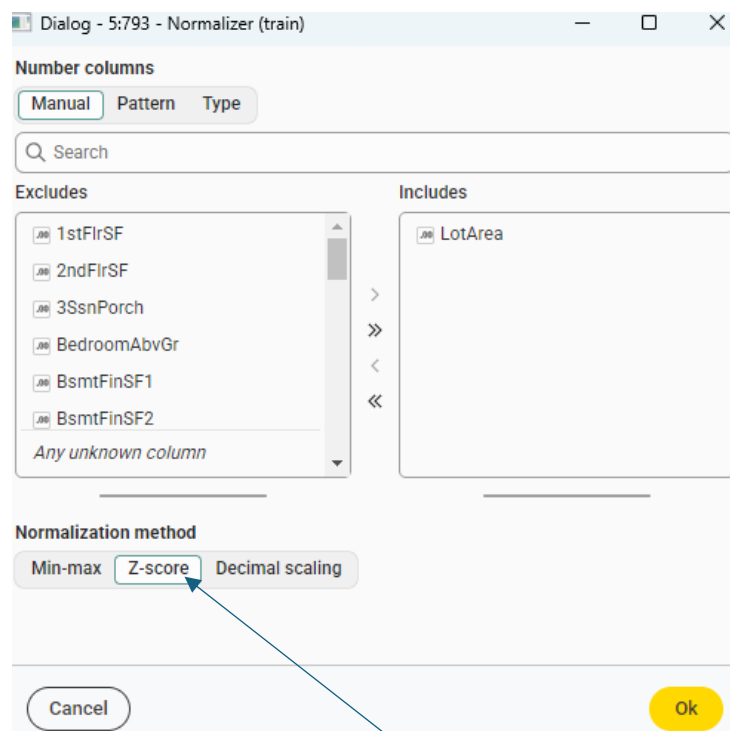


Figure 15: Select z-score

And here is the window of Rule-based row filter.

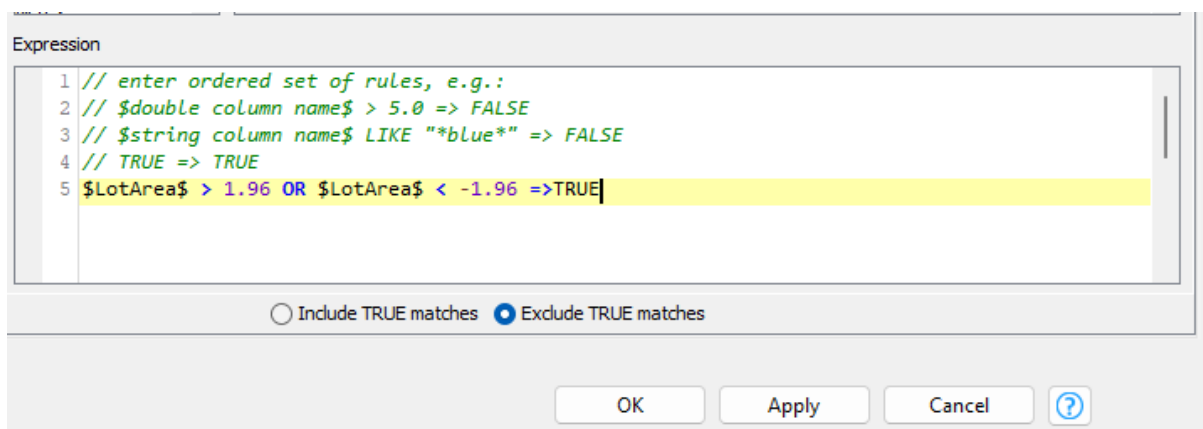


Figure 16: Exclude all rows that match the condition

Chapter-VI

Modelling

File: outlierRemoval_ImprovingXgboost-II

Perform modelling again after creating another metanode as below:

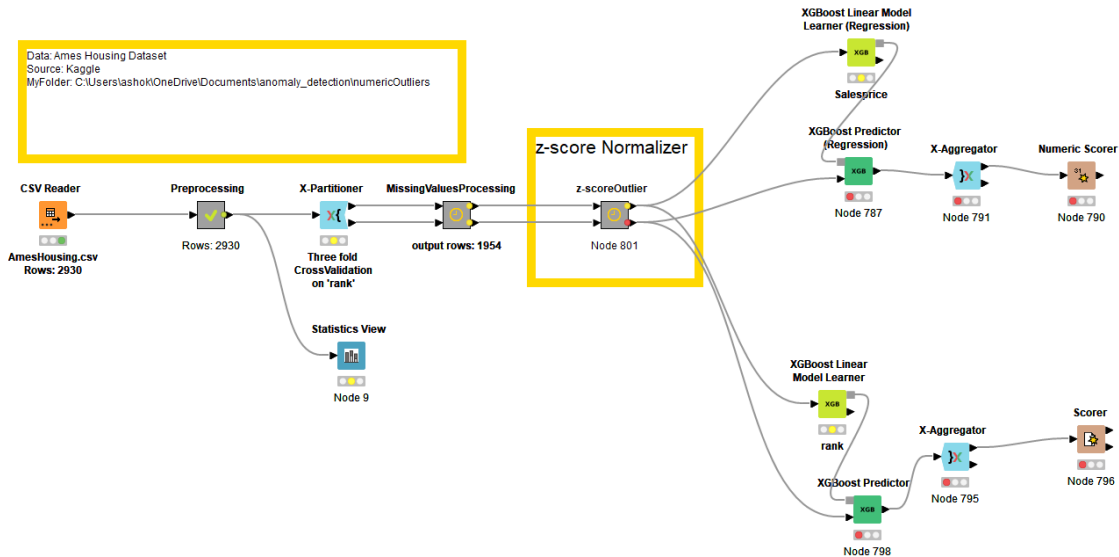


Figure 17: We get better results for Saleprice prediction

Result for 10-fold cross validation is as below:

Statistics - 8:790 - Numeric Scorer	
File	
R ² :	0.909
Mean absolute error:	16,045.138
Mean squared error:	563,947,913.519
Root mean squared error:	23,747.588
Mean signed difference:	-216.265
Mean absolute percentage error:	0.1
Adjusted R ² :	0.909

Chapter VII

Numeric Outlier

We further add numeric outlier (based on IQR metrics) as below:

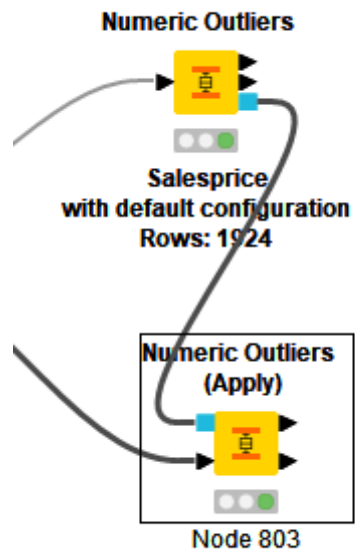


Figure 18: Salesprice with numeric outlier (IQR)

Dialog - 7:802 - Numeric Outliers (Salesprice)

Outlier selection

Manual Pattern

Search

Excludes

- 1stFirSF
- 2ndFirSF
- 3SsnPorch
- BedroomAbvGr
- BsmtFinSF1
- BsmtFinSF2
- Any unknown column

Includes

- SalePrice

General Settings

Interquartile range multiplier (k)

1.5

Quartile calculation

Full data estimate Use heuristic

Estimation type

R 6

☐ Update domain

Outlier Treatment

Apply to

All outliers

Treatment option

Remove outlier rows

Group Settings

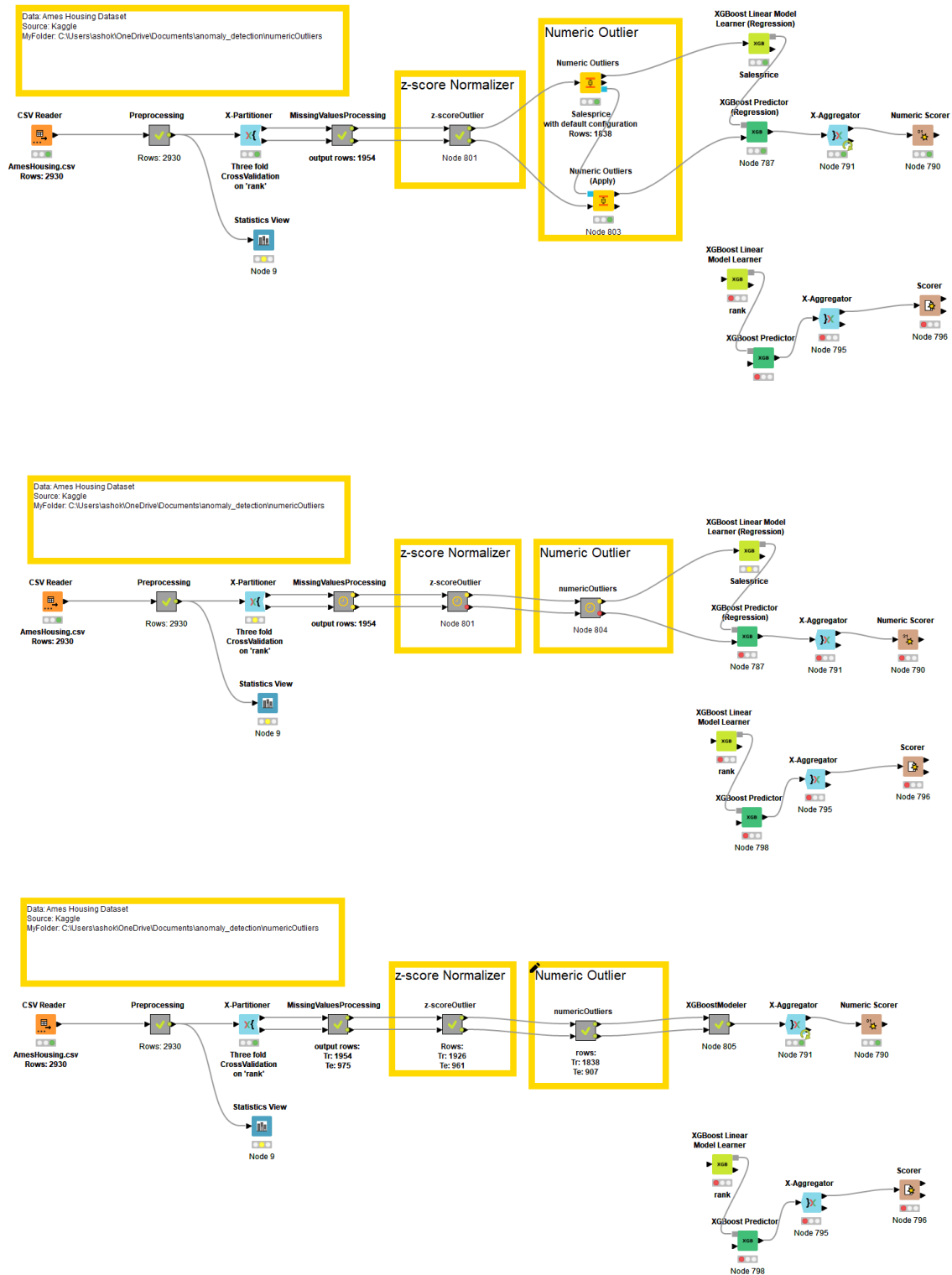
Cancel Ok

Figure 19: For Treatment option, Remove outlier rows

Chapter VIII

Modelling

File: outlierRemoval_ImprovingXgboost-III

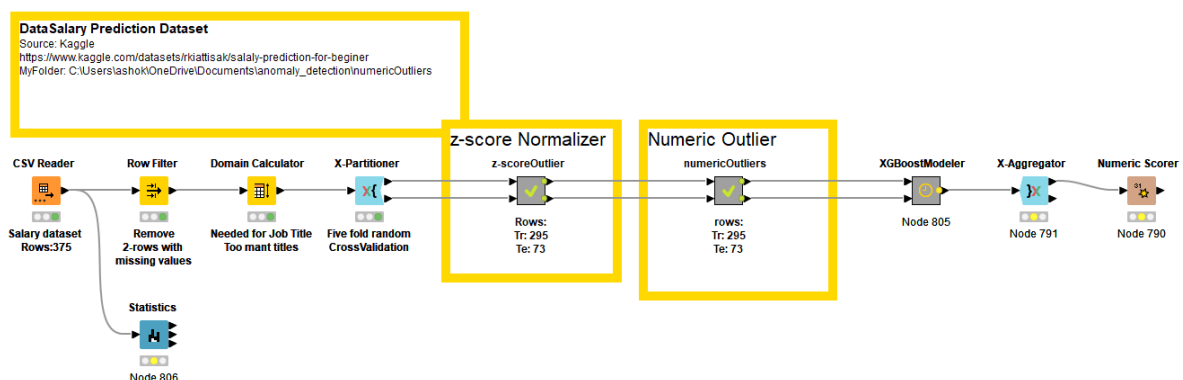


Results progressively get better Chapter by Chapter, especially for *SalesPrice* target. Results for 10-fold cross-validation are as below:

Statistics - 7:790 - Numeric Scorer	
File	
R ² :	0.888
Mean absolute error:	13,940.946
Mean squared error:	386,402,149.316
Root mean squared error:	19,657.114
Mean signed difference:	-90.352
Mean absolute percentage error:	0.094
Adjusted R ² :	0.888

Salary Dataset

It's a small dataset. We have to predict salary. There are only two rows with missing data. We remove them. Z-score outlier filtering is done on Age and IQR filtering is done on *Salary*. Possibly IQR filtering is not removing any rows. You can also try with z-score filtering on Salary to see how much performance improves. We use Random Sampling with 5-fold cross validation. The workflow is simple and is as below. Job Title has multiple levels. We need a *Domain Calculator* to understand it.



Outlier removal does make marginal improvements, dataset being small.

More Datasets

In Kaggle, search for datasets on Regression problems. OR on Internet make the same search. The above modelling approach will apply to most problems on regression.

#####